StudyVance AI Explorers - Level 1

Development Scope and Build Specification for Z.ia

StudyVance AI Academy

October 2026

Scope at a glance

Build goal: A real, working, child-friendly mission application - not a static mock-up or a set of lesson pages.
Learners: Grades 4-6, approximately ages 9-12.
Learning model: Self-paced audio/visual lessons, interactive games, retry feedback and a tutor on standby.
Curriculum: Six missions, aligned to the supplied StudyVance Teacher Guide and Learner Workbook.
Login: Learner code + short PIN; no student email or long password.
Platform: Installable offline-capable web app for Windows, backed by the existing StudyVance WordPress site for online progress and class management.
Cost principle: Free/open-source components and no paid AI API at runtime. Existing hosting and maintenance costs are not assumed to be zero.

First milestone: Deliver and test Mission 1 with student-specific progress saved online/offline before building the remaining five missions.

1. Instruction to the AI coding agent

Build a real, working, child-friendly educational mission application-not just a visual mock-up, static landing page, prototype screenshots or an LMS course outline. The application must deliver the Level 1 learning experience, save progress per learner, function offline after its content and roster have been prepared on a device, and synchronise progress with the StudyVance server when online.

Before coding:

Inspect the supplied project/repository and the reference files listed in Section 3.

Report the existing stack, deployment constraints and any relevant code you can reuse.

Propose a short implementation plan and identify assumptions. Do not wait for approval for routine reversible work; do not change the live production site, delete data, add paid services, or introduce recurring fees without explicit approval.

Implement in small working slices, starting with student profile login, Mission 1, local save and offline behaviour. Test each slice before expanding.

Use the curriculum in this specification and the supplied guides as the source of truth. Do not replace or substantially rewrite the learning objectives without recording the change in the handover notes.

Do not claim any feature is complete unless it is implemented and tested. Do not leave fake buttons, empty interactions, placeholder lessons, broken media paths or simulated progress in the final MVP.

If something cannot be completed because of the hosting environment, describe the blocker precisely, implement the closest safe working alternative, and document the remaining limitation. Do not secretly switch to a paid service or a cloud-only architecture.

2. Product vision

Children enter a friendly “AI Explorer Academy” led by a recurring robot guide. They move through six missions. Each mission has a short narrated lesson, a demonstration, an interactive game or puzzle, a quick knowledge check, a practical activity, and a reward. Their progress is saved under their own learner profile. The tutor is available to help if needed but does not need to deliver the lesson live.

The learning experience should feel like a polished educational adventure-not a worksheet pasted into a website and not a complex commercial video game.

Success criteria

A Grade 4 learner can understand what to do next without an adult explaining every screen.

A learner can replay or reread instructions, make mistakes safely, receive useful feedback and retry.

The core application does not need a live generative-AI connection, paid AI API, camera or microphone.

The six missions can be completed using prepared examples and included media.

A student’s progress remains separate from other students’ progress on a shared computer.

Progress remains saved when the app is closed and reopened.

After a device has been prepared for offline use, lessons and activities work without internet; offline progress synchronises when connectivity returns.

A tutor/admin can create learners, reset a PIN, see progress and export a simple report.

3. Reference material and source-of-truth rules

Use the files supplied with this brief:

StudyVance_Level_1_Teacher_Guide.pdf - the six lesson plans, prepared demonstrations, answer guidance, assessment rubric and parent information.

StudyVance_Level_1_Learner_Workbook.pdf - learner activities, Moon fact sheet, prompts, story project and independent skills check.

StudyVance_AI_Academy.pdf - the original proposed academy structure and six-session beginner pathway.

IMG-20261014-WA0020.jpg - the StudyVance AI Explorers promotional artwork / visual branding.

If included: current StudyVance Tutor LMS lesson pack and learner passport. These are supporting implementation references, not a reason to duplicate the whole LMS inside the app.

The Teacher Guide says the course is for Grade 4-6 beginners, uses six practical lessons, has a 60-minute core plus an optional 15-minute extension, and does not require individual learner AI accounts. Teachers may explain orally in Afrikaans while this edition remains in English. Use the supplied example texts honestly: prepared examples are instructional texts, not real AI outputs unless a live teacher-operated demonstration actually occurred.

Preserve the learning outcomes and facts from the source documents. If a source item is ambiguous, flag it in the README/handover instead of silently inventing a replacement.

4. Platform and architecture

Preferred architecture

Build an installable Progressive Web App (PWA) served from the StudyVance WordPress site, ideally at a dedicated route such as /ai-explorers/, plus a small custom WordPress plugin for the authenticated API, learner roster, progress records and tutor/admin dashboard.

Front end: lightweight HTML/CSS/JavaScript or a comparably small maintainable framework. Choose one stack and document it.

Offline app shell and static lesson assets: service worker + Cache Storage.

Structured local progress, queued events and cached offline roster: IndexedDB (not localStorage for the main database).

Online server: existing WordPress installation and its supported database, through a custom namespaced REST API and custom database tables where appropriate.

Admin access: authenticated WordPress administrator/tutor account with role/capability checks.

Student access: a separate learner-profile mechanism described in Section 5; do not create WordPress dashboard accounts for children unless explicitly justified and approved.

Keep app and API on the same origin where possible to simplify cookies, CORS and PWA scope.

Avoid large frameworks and unnecessary libraries. Prefer maintained free/open-source dependencies and pin versions.

Offline expectations

A PWA must be loaded at least once online or distributed with a supported pre-install method before it can be expected to run offline. Build a clear “Prepare this computer for offline use” workflow that downloads/caches the full Level 1 content and the selected class roster.

When offline:

load the cached app, all six mission shells, lesson text, images, required fonts/icons, quiz data and all bundled audio/video assets;

let a previously prepared learner sign in locally and continue;

write progress events and answers to IndexedDB immediately;

display a clear “Saved on this computer - waiting to sync” status;

provide a manual “Sync now” action when online returns; also retry sync automatically when connectivity returns and the app is open;

never lose work or show a blank/error page because a request failed;

never pretend the server is updated until the server confirms the sync.

Offline login on a device is available only for learner profiles previously downloaded/prepared on that device. First-time creation/login and cross-device progress sync require an online connection. State this clearly to staff.

If WordPress hosting makes a same-origin PWA impractical, first document the exact blocker and propose a local desktop wrapper or alternative. Do not move straight to Electron/Tauri or a paid hosted backend without explaining the trade-offs and getting approval.

Do not use in the MVP

No live generative-AI API or per-token model calls.

No external YouTube/Vimeo embeds or remote media that breaks offline play.

No advertising, third-party tracking analytics, public leaderboard or social sharing.

No mandatory camera, microphone, account with another AI provider, external API key or paid plugin.

No 3D world, multiplayer, chat platform or complex game engine.

No dependency on Tutor LMS Pro to make the game itself work. The app may link to Tutor LMS, but must function as a standalone learning experience.

5. Student login and progress model

Recommended low-friction login

Use a **learner code** + **short PIN**, not an email address or conventional long password.

A tutor/admin creates each learner record and assigns a pseudonymous display name/nickname, grade, cohort and an automatically generated non-sequential learner code (example format: EX-7K4P).

Each learner has a 4-digit PIN for simple classroom use. Make PIN length configurable; default to 4 digits for the pilot. Do not use birth dates, phone numbers or predictable PINs such as 1234 as a default. The tutor gives the learner the code/PIN privately and can reset the PIN.

A learner can select a friendly avatar after successful login. The avatar is decorative and is not a security factor.

No student email, phone number, full date of birth or full legal name is required.

Teacher/admin accounts use normal protected WordPress login and strong credentials; never give learners an admin/tutor login.

Add explicit **Log out / Switch learner** controls. Warn before switching if the current learner has unsaved work.

A PIN is a convenient profile-separation mechanism for a classroom, not strong protection against a person with physical access to the computer. Do not store the PIN in plain text. Hash it server-side using WordPress's supported password hashing/checking functions or an equivalently secure implementation. Rate-limit failed online attempts and temporarily lock repeated attempts. For offline use, cache only a salted verifier needed for local PIN checking; explain in technical notes that local authentication is a convenience barrier, not a high-security boundary. Never store a raw PIN in IndexedDB, localStorage, logs or analytics.

Offline roster preparation

The tutor/admin must be able to select a cohort and prepare a computer for offline use. It should cache:

the full Level 1 content and media;

the selected cohort's minimal learner codes, display names/nicknames and avatar choice;

only the local salted PIN verifiers needed for offline unlock;

each cached learner's latest known progress version or revision marker.

Do not cache tutor/admin credentials or allow offline tutor/admin functions. Provide a "Remove this class from this computer" control that deletes cached roster data and local learner data after warning and asking for confirmation. Warn that any unsynchronised progress will be lost if erased, and offer export/sync first where possible.

Student isolation

Learner A must never be able to view or modify Learner B's progress, saved story or quiz history through normal app navigation or API requests.

Switching learner clears all learner-specific screen state before displaying another learner.

Every online API request must verify that a learner can access only their own records. Never trust a learner_id supplied by the client alone.

Tutors/admins can view only the groups and records authorised for their role.

6. Student experience and visual design

Brand

Use the supplied artwork as brand reference: deep navy blue, turquoise/cyan, bright yellow, white and friendly illustrated characters. Keep the robot guide recurring across all missions. Use consistent icons, buttons, progress indicators and badges. Do not redesign the existing StudyVance logo.

Core screens

Welcome screen: StudyVance AI Explorers, short friendly greeting, "Start exploring" and sound/accessibility controls.

Sign-in screen: learner code + PIN; show/offline status; no email field; plain help hint "Ask your tutor if you forgot your code or PIN."

Choose avatar: a small set of inclusive static avatars; no upload/photo capture.

Mission map: six large mission cards, visible progress, locked/unlocked/completed state and earned badges.

Lesson player: narrator/audio player, illustration/video or animated scenes, captions/transcript, replay, previous/next, progress state and tutor-help button.

Activity/game screen: one task per screen, clear directions, large targets, keyboard-and-mouse controls, non-colour-only feedback and repeatable attempts.

Feedback screen: explain why an answer works or does not; provide a retry or hint. Never shame or punish learners for mistakes.

Explorer portfolio: story plan, story drafts, drawing and truthful assistance note. Save continuously.

Progress/reward screen: mission completions, badges and a simple "what you can now do" recap. No public competition or leaderboard.

Tutor/admin dashboard: roster, progress and quiz results, help flags, roster management, PIN reset, offline preparation and CSV export.

Age and accessibility design

Use plain English suitable for Grades 4-6; short paragraphs and one instruction at a time.

Prefer a narrated 2-4 minute visual explanation per mission, broken into scenes rather than a long lecture.

Bundle audio locally (MP3/OGG or equivalent supported format); include replay, pause and visible transcript/subtitles. Do not auto-play sound.

Do not rely on browser text-to-speech for the required narration because voice availability may vary offline. Browser speech synthesis may be an optional backup only.

Provide keyboard navigation, visible focus, readable contrast, scalable text, reduced-motion support, mute control and alt text for meaningful images.

Keep animations short and optional. Respect prefers-reduced-motion.

Provide a way for the tutor to read instructions aloud without changing the learner's response or score.

Avoid timed pressure as a requirement. Optional fun animations must not block the next action.

7. Curriculum specification: six missions

Each mission must be self-paced and include: (a) mission objective in child-friendly words; (b) short narration/illustrated explanation; (c) example/demo; (d) an interactive task; (e) a quick quiz of five questions; (f) encouraging feedback and retries; (g) a practical workbook/portfolio activity; (h) an optional home challenge; and (i) recorded progress. Target about 35-50 minutes of on-screen and workbook time combined, with an optional 10-15 minute extension. Learners may pause and resume.

Mission 1 - Meet AI

Learner-facing hook: “Can a computer learn to recognise a cat?”
Outcomes: explain AI simply as software that learns patterns from examples; identify that AI can be useful but wrong; apply a privacy rule.
Lesson sequence: welcome and baseline question → narrated explanation → sort examples → privacy challenge → “confidence is not proof” mini puzzle → five-question check → reflection.
Interactive game: drag/tap four cards into “AI example” and “ordinary programmed behaviour” (can also be completed with keyboard): basic calculator follows fixed steps = ordinary programmed behaviour for this example; kitchen timer = ordinary programmed behaviour; model trained on labelled animal images predicts cat = AI example; text system learns patterns and creates a story = AI example. Explain that a modern product can combine ordinary software and AI features.
Privacy game: remove real name, home address, phone number, password and another child's identifying photo from unsafe fictional prompts. Teach use of fictional names and respectful tasks.
Check/feedback: include the misconception “AI is always right because computers never make mistakes.” Correct feedback: AI can make mistakes and a confident answer is not proof.
Portfolio/home task: “AI can… / AI may… / One privacy rule is…” Optional family task: explain one useful AI activity and one rule; no account needed.

Mission 2 - Ask Better Questions

Learner-facing hook: “Can you make this instruction clearer?”
Outcomes: identify the task, useful details and requested format in a prompt; improve vague instructions without private information; compare the output to the request.
Teaching recipe: TASK + DETAILS + FORMAT. A prompt is a question or instruction given to an AI tool. More words are not automatically better; useful details matter.
Interactive game: assemble a prompt from three parts and improve “Tell me about space” or “Make a story.” Example target: “Write three short sentences for a Grade 4 reader about a fictional robot rabbit helping an animal shelter. Use simple words and a friendly tone. Do not include real names.”
Compare: reply A: “Animals are found in many places. They can be large or small. People like animals.” Reply B: “A robot rabbit carried blankets to the animal shelter. It helped the keeper fill the water bowls. At sunset, every animal had a cosy place to rest.” Ask which reply follows the request better and why. Also test output format: a paragraph does not meet a request for three bullet points.
Portfolio/home task: create a prompt with task, detail and format; do not enter personal information. Optional family activity: write clear paper instructions for making a sandwich.

Mission 3 - Become an AI Detective

Learner-facing hook: “The computer says it is true. Is that enough?”
Outcomes: distinguish claim from evidence; use supplied evidence to check a claim; label a claim as supported, contradicted or not enough information; correct an answer.
Interactive game: use the supplied Moon fact sheet: (A) the Moon reflects sunlight; (B) the Moon orbits

Earth; (C) Earth has one natural moon; (D) the Moon has no breathable atmosphere for people; people need life-support equipment. Test claims: Moon makes its own light = contradicted (A); Moon orbits Earth = supported (B); Earth has two natural moons = contradicted (C); people can breathe normally on the Moon without equipment = contradicted (D); a purple dragon lives on the Moon = not enough information from this sheet. Label the dragon as fiction rather than fact.
Teach: Read → Compare → Decide → Correct. Asking the same AI again is not independent proof.
Portfolio/home task: rewrite the flawed Moon paragraph using only supported facts. Optional family task uses a non-personal textbook fact, not compulsory web research.

Mission 4 - Create Something

Learner-facing hook: "Can you rescue this story?"
Outcomes: plan a fictional character, setting, problem and kind solution; review a prepared draft against a brief; make purposeful revisions and identify personal contribution.
Activity: fill the four story-planning fields. Present the supplied practice draft: "Pip the robot rabbit worked at an animal shelter. One morning, a kitten's blanket was missing. Pip shouted at the kitten for losing it. Then a magic wand appeared and made a hundred blankets. The kitten flew into space. Everyone clapped." Shared practice brief: kind story at an animal shelter, no magic. Ask learners to spot where the draft breaks the brief.
Create: write or improve a 6-8 sentence fictional story for a Grade 4 reader, including character, setting, problem and kind solution; draw at least one matching illustration. Learners may use the canvas or complete the supplied printable workbook. Drawing quality is not graded.
Assistance note: truthfully select prepared practice draft, teacher-operated live AI, or own first draft without AI; explain help and own contribution. The app must never claim AI was used if it was not.
Portfolio/home task: plan/finish a scene or tell the story to family; no learner AI account.

Mission 5 - Learn with AI

Learner-facing hook: "Should AI give you the answer or help you find it?"
Outcomes: ask for a hint/explanation; check a proposed method; solve a similar problem independently; support reading answers with evidence from a passage.
Activity A: compare "Give me the complete answer to copy" with "Give me one hint, then let me try." Use equal groups for 24 ÷ 3. Correct answer = 8, checked by 3 × 8 = 24. Correct the deliberately wrong explanation "24 ÷ 3 = 7 because 3 × 7 = 24" (3 × 7 = 21). Independent check: 18 ÷ 3 = 6.
Activity B: passage: "Lebo took a small watering can to the school garden. The soil near the beans felt dry. Lebo watered the soil slowly, then placed a label beside each row. Before leaving, Lebo closed the gate to keep the goats out." Questions: what felt dry; why close the gate; true/false that the gate was left open; whether the passage says the can was red (not enough information).
Portfolio/home task: state one way a hint helped and one skill completed independently. Optional family task: solve 12 ÷ 3 on paper.

Mission 6 - Finish and Present

Learner-facing hook: "Show what you can create!"
Outcomes: complete the story; explain a revision; truthfully acknowledge assistance; demonstrate Level 1 skills independently.
Project requirements: 6-8 sentences, fictional character, setting, problem and kind solution; at least one matching learner-created drawing; retain first draft and final draft; improve at least two sentences and explain one change; include an accurate assistance note; no real personal details.
Independent skills check: five items covering (1) AI can make mistakes; (2) safe prompt choice versus a password; (3) improve "Make a story" using task/details/format; (4) correct the two-moons claim using the supplied fact; (5) explain why a hint helps more than copying.
Rubric: clear instructions; checking; story and revision; privacy; own contribution. Each skill 0 = not yet demonstrated, 1 = demonstrated with support, 2 = demonstrated independently. Suggested Level 2 readiness from the guide: 8/10 or higher, privacy = 2, and at least 4/5 on the independent check. Keep readiness separate from course completion and allow tutor override/review. Do not claim accreditation.

Outcome: learner exports/prints or shows their story portfolio and earns an “AI Explorer” completion badge/certificate. Tutor can mark “ready for next level” or “more practice” after reviewing evidence.

8. Game mechanics and reward principles

Six-mission map with progress indicators.

Earn a badge after a mission is completed; badge records the skill, not only a score.

Show success, retries and hints in a supportive way.

Let learners resume where they stopped.

Use small celebratory animations/sounds, all optional and muted by default if appropriate.

Do not show public scores, rank children, punish mistakes, remove earned badges, or require speed to pass.

Unlock the next mission after the learner has completed core activities and passed that mission's quiz at 4/5 or better. A learner may retry without a penalty. For privacy-critical items, require a correct answer before completing the relevant mission; explain the privacy rule and allow another attempt.

Mission 6 completion does not automatically declare readiness for Level 2. Tutor reviews the rubric/independent check.

9. Audio, video and media requirements

Every mission has a written narration script plus bundled, prerecorded audio; include visual slides/scenes and captions/transcript. Add a short video/animated sequence where feasible, but don't delay the working MVP for cinematic animation.

Each narrated introduction should normally be 2-4 minutes, segmented into short scenes. Provide replay, pause and transcript.

Media must live with the app or be downloadable for offline use. Do not stream from third-party platforms.

Provide compressed assets appropriate for a school internet connection. Show download/preparation progress and estimated content size before offline caching.

Create an asset manifest so missing audio/image/video files can be detected during testing.

Do not use generated media that depicts a child as an identifiable learner. Use fictional, licensed or original generic illustration.

Use English audio for this pilot. Keep interface text in a way that can later be translated into Afrikaans without rewriting the application architecture; translated audio can be a later version.

10. Learner progress, data and tutor/admin controls

What to store

Minimum required fields:

internal random learner ID;

learner code;

pseudonymous display name/nickname;

grade band and cohort;

PIN hash (never plain text) and account active/inactive state;

mission/lesson completion events;

quiz attempts and scores;

earned badges;

story project text/drawing only if the site operator decides to sync and retain it, with parent-facing notice/consent; otherwise it remains local/exportable;

last successful sync timestamp and app/content version.

Avoid collecting date of birth, personal email, phone, home address, photo, voice recording, precise location, advertising ID or unrelated behavioural tracking.

Tutor/admin functions

Login through protected WordPress admin authentication; separate capabilities for tutors and administrators.

Create, edit, deactivate and delete learner profiles.

Create/manage cohorts and assign learners.

Generate/reprint learner code; reset PIN and record that it was reset without exposing the old PIN.

Prepare a selected cohort/device for offline use and view cache status.

View each learner's mission completion, latest/best quiz score, badges and sync status.

View a “needs tutor help” flag with the mission and a learner-selected help reason. The button need not create live chat.

View/review final project if the learner opted to sync it and the configured consent permits it.

Export a cohort/learner progress report as CSV.

Clear cached cohort from a device and handle account deletion/data export in accordance with the site owner's policy.

Progress sync

Use uniquely identified progress events with a stable event ID so retries don't create duplicates.

Queue events offline in IndexedDB and sync them when online.

Server validates each event and learner ownership, stores events idempotently and returns a confirmed sync result.

Merge rules: completion is monotonic; preserve the best/latest quiz attempt fields clearly; never silently overwrite the story with an older offline version. If two story edits conflict, keep both versions or ask the tutor to resolve.

Show states: “Saved on this computer”, “Waiting to sync”, and “Synced”.

Provide “Sync now” and last-sync time to the tutor.

11. API and WordPress implementation outline

Use a namespaced API, e.g. /wp-json/studyvance/v1/. Adjust names if an existing project convention requires it, but document the final API.

Suggested endpoint responsibilities:

student login and logout;

retrieve current learner profile and own progress;

retrieve versioned content/roster manifest for offline preparation;

submit a batch of progress events idempotently;

submit/retrieve a learner's project if enabled;

post a help request/flag;

admin-only roster/cohort CRUD, PIN reset, sync health and CSV export.

Security requirements:

same-origin HTTPS for online operation;

secure session mechanism; if using cookies, use HttpOnly, Secure and appropriate SameSite settings with CSRF protection;

role/capability checks on every tutor/admin endpoint;

parameter validation/sanitisation, prepared database queries and output escaping;

online PIN rate limiting and temporary throttling/lockout after repeated failures;

no secrets, salts, database passwords, tokens or private keys in source control or frontend code;

never store server auth tokens in localStorage;

do not expose personal data, PIN hash, or one learner's records in another learner's API response;

log security-relevant events without logging PINs or sensitive learner content.

If the existing WordPress environment cannot support the proposed implementation, document a concrete migration recommendation. Do not use an insecure “secret URL = authentication” design.

12. Data protection and child safety

The product is for children. Apply data minimisation and privacy by design. The site owner must confirm the appropriate South African POPIA notice, parent/competent-person consent process and retention/deletion policy before the live pilot; this specification is not legal advice. The scope intentionally avoids collecting direct identifiers where possible.

No student accounts for ChatGPT or other external AI are required.

The app does not send student text to an AI provider.

The app labels prepared examples as prepared examples, not genuine live AI output.

Use fictional scenarios and remind learners not to enter passwords, addresses, phone numbers, private photos or identifying information.

Tutor access is limited to authorised staff; learner records are not public.

Do not publish stories/drawings publicly or share them outside the cohort without appropriate permission.

Add a parent/tutor information page explaining what is stored, why it is stored, who can see it and how deletion/export is requested.

Provide account deactivation and removal; document what happens to server data and cached offline copies.

13. Suggested data model

Use a design suitable for WordPress MySQL or the chosen backend. Names below are conceptual; adjust to the codebase.

cohorts: id, name, grade_band, active, created_at.

learners: random id, cohort_id, learner_code, nickname, grade, avatar_id, pin_hash, active, created_at, updated_at.

progress_events: event_id (unique), learner_id, mission_id, activity_id, event_type, score, max_score, payload_json (strictly limited), occurred_at_client, received_at_server, app_version.

learner_state: learner_id, mission completion summary, best/latest quiz results, badge list, content version, last sync.

learner_projects (optional synced projects): project id, learner_id, version, story_text, drawing_data or stored file reference, assistance_note, updated_at.

tutor_help_flags: id, learner_id, mission_id, help_category, created_at, resolved_at, resolved_by.

Never use the PIN or learner code as the database primary key. Index cohort, learner and event IDs appropriately.

14. Required deliverables from Z.ia

Working application source code in a clean, documented repository.

Installable PWA and all six missions.

Custom WordPress plugin or documented backend providing authentication, learner records, progress sync and tutor/admin dashboard.

Bundled narration scripts, audio files, illustrations, captions/transcripts and asset manifest for all six missions.

Student code + PIN login, teacher roster management, reset/deactivation and device offline preparation.

IndexedDB offline storage and idempotent online sync.

Six functional games/activities and six quizzes with at least five questions each; feedback and retries.

Learner portfolio/story editor and drawing area or a supported printable workbook path.

Unit/integration tests and a repeatable smoke-test checklist.

Installation/deployment instructions for the existing WordPress hosting, HTTPS requirements, backups, updates and rollback.

Teacher/admin guide: add learner, prepare offline device, reset PIN, view progress, export report, sync offline work and remove a cohort from a device.

Parent-facing data/privacy summary draft and statement of what the app collects.

A README.md, .env.example if needed, dependency/license list, asset attribution list, known limitations and screenshots.

A ZIP release bundle that can be installed/tested on a Windows computer.

Do not put production credentials in the ZIP or repository. Do not make the live site changes without explicit permission.

15. Acceptance tests - MVP is not complete until these pass

Core gameplay

☐ Welcome/sign-in/missions screens are functional, responsive and styled to StudyVance branding.

☐ All six missions have readable content, narration audio, captions/transcripts, activity, a five-question quiz, feedback, retry and completion state.

☐ Mission progression follows the intended order; a learner can resume after closing the app.

☐ Story editor/portfolio can save a 6-8 sentence story, a drawing or equivalent illustration, first/final revision and truthful assistance note.

☐ All important actions work by mouse and keyboard; no required camera/microphone.

Identity/progress

☐ An admin can create two test learners without email addresses.

☐ Learner A and Learner B get distinct codes/PINs and their progress does not mix.

☐ Incorrect PIN attempts are throttled; tutor can reset a PIN.

☐ Learner A cannot inspect or change Learner B's data by changing a URL/request or editing the browser UI.

☐ Progress, quizzes, badges and project drafts survive app close/reopen.

☐ Admin can see progress and export a CSV.

Offline and sync

☐ After online setup/content download, disable internet and restart the app: sign-in for a preloaded learner, all six lessons, all audio, games, quizzes and progress saving still work.

☐ No required asset is remotely streamed after offline preparation.

☐ While offline, progress is marked waiting to sync; no false “synced” confirmation appears.

☐ Restore connectivity and sync: server receives progress; repeating sync does not duplicate events.

☐ An online login from another computer sees previously synced progress.

☐ If a learner is not preloaded on an offline device, the app explains that the tutor must prepare the device or connect online; it does not silently create a duplicate profile.

☐ Removing a local cohort warns about unsynced progress and then clears the selected local data after confirmation.

Quality and safety

☐ No broken links, missing media, console errors blocking progress, dead buttons, or unresolved placeholders.

☐ No live AI API, paid service, ads or third-party analytics is called.

☐ No PIN or auth token is stored in plain text in client storage/logs.

☐ Learner-facing content does not expose tutor answer-key pages or admin reports through normal app navigation.

☐ UI includes transcripts, focus states, readable contrast, mute/replay, and non-colour-only signals.

☐ POPIA notice/consent and retention decisions are flagged as required pre-launch tasks.

☐ Test report records browser/OS, online/offline steps, results, known limitations and version number.

16. Delivery phases

Phase 0 - Inspect and plan
Confirm repository, hosting, WordPress version, PHP version, database permissions, current theme/plugin conflicts and deployment method. Report a proposed architecture and risk list. No production changes.

Phase 1 - Working vertical slice
Implement PWA shell, branding, one learner code + PIN login flow, one learner record, Mission 1 narrated lesson and sorting game, five-question quiz, local progress storage and offline smoke test. Provide screenshots and test notes.

Phase 2 - Backend and tutor tools
Add server authentication, roster/cohort management, PIN resets, protected progress APIs, tutor dashboard and CSV export; confirm two accounts remain isolated.

Phase 3 - Complete curriculum
Build Missions 2-6 using the content requirements above, all narration/media and quiz feedback, story editor/drawing/portfolio and completion/readiness workflow.

Phase 4 - Offline and sync hardening
Roster/content preloading, progress event queue, duplicate-safe sync, conflict handling, cache removal and offline UI states.

Phase 5 - QA and packaging
Run all acceptance tests, test on the actual classroom computers, fix defects, package a Windows-friendly PWA install kit, write the staff guide and prepare a release ZIP.

At the end of each phase, report: what is complete, what was tested, test results, files changed, known issues and the next phase. Prefer a small working vertical slice over building all screens as placeholders.

17. Explicit non-goals for the pilot

Levels 2-5 of the wider AI Academy.

Student accounts on external AI platforms.

Real-time AI image, text, audio or video generation.

User-uploaded photos, voice recording or webcam/microphone features.

Public ranking, multiplayer, social sharing or parent mobile app.

Automated decisions about a learner's ability or school achievement.

Accredited certificates or claims that completion represents professional AI competence.

Full native Windows installer unless the PWA cannot meet an agreed deployment requirement.

18. Technical reference links

Use current documentation before implementing details; validate against the actual hosting and browser versions.

MDN: Progressive Web Apps and offline operation:
https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Offline_and_background_operationX

MDN: PWA reference, Cache Storage and IndexedDB overview:
https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/ReferenceX

WordPress REST API authentication:
https://developer.wordpress.org/rest-api/using-the-rest-api/authentication/X

WordPress REST API handbook: https://developer.wordpress.org/rest-api/X

OWASP Authentication Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.htmlX

OWASP Session Management Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.htmlX

South African POPIA, sections 34-35 on children's personal information:
https://www.justice.gov.za/legislation/acts/2013-004.pdfX

Final instruction

Build the MVP to this scope, keeping it easy for a child to use, safe for classroom devices, and practical for the existing StudyVance setup. The first priority is a genuinely working Mission 1 with student-specific progress and offline operation. Once that vertical slice passes the acceptance checks, complete the remaining five missions in the same consistent framework.